Grafana Alloy

Automatisation du déploiement via Ansible

Collecte centralisée des logs vers Loki

Projet IUTO — Documentation technique
Avril 2026

Table des matières

Table des matières 1
Contexte et objectif 1
Serveurs concernés 1
Prérequis 1
Sur le contrôleur Ansible (SRV-ANS) 1
Sur chaque serveur cible 1
Pare-feu Stormshield 1
Structure du projet Ansible 1
Configuration complète des fichiers 1
ansible.cfg 1
inventory/prod.yml 1
playbook.yml 1
roles/alloy/tasks/main.yml 1
roles/alloy/handlers/main.yml 1
roles/alloy/templates/config.alloy.j2 1
Exécution du déploiement 1
Vérification post-déploiement 1
Configuration manuelle des serveurs Windows 1
Installation 1
Configuration 1
Serveurs Windows du projet 1
Erreurs rencontrées 1
Commandes utiles 1

Contexte et objectif

Dans le cadre du projet IUTO, l'infrastructure est composée de multiples serveurs Linux et Windows. Pour centraliser les logs de tous ces serveurs vers Grafana Loki, nous déployons Grafana Alloy (successeur de Promtail) comme agent de collecte sur chaque machine.

Plutôt que d'installer et configurer Alloy manuellement sur chaque serveur un par un, nous utilisons Ansible pour automatiser ce déploiement. En une seule commande, Ansible installe Alloy, dépose la configuration adaptée à chaque serveur, et démarre le service sur tous les serveurs Linux du parc.

Serveurs concernés

Serveur	IP	OS	Logs collectés
SRV-ANS	10.200.1.70	Debian 13	systemd, syslog, auth, kernel
SRV-CLOUD	192.168.1.1	Debian 13	systemd, syslog, auth, kernel + Docker
SRV-FOG	10.200.1.60	Debian 13	systemd, syslog, auth, kernel
SRV-GLPI	10.200.1.120	Debian 13	systemd, syslog, auth, kernel
SRV-XDR	10.200.1.40	Debian 12	systemd, syslog, auth, kernel
SRV-ZABBIX	10.200.1.45	Debian 13	systemd, syslog, auth, kernel

SRV-CLOUD dispose en plus de la collecte des logs Docker (conteneurs Nextcloud, Collabora, MariaDB, Redis) grâce à la variable collect_docker: true.

Le serveur de logs SRV-LOG (10.200.1.80) héberge Loki et Grafana et n'a pas besoin d'Alloy via Ansible car il est déjà configuré.

Prérequis

Sur le contrôleur Ansible (SRV-ANS)

— Ansible installé (apt install ansible)

— Clé SSH générée et copiée sur chaque serveur cible

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ssh-copy-id sio@10.200.1.60
ssh-copy-id sio@10.200.1.120
ssh-copy-id sio@10.200.1.45
ssh-copy-id sio@10.200.1.40
ssh-copy-id sio@192.168.1.1
```

Sur chaque serveur cible

— Utilisateur sio avec accès sudo sans mot de passe

— Python 3 installé (présent par défaut sur Debian)

— Serveur SSH actif

```
apt install -y sudo openssh-server
echo 'sio ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/sio
```

Pare-feu Stormshield

Pour SRV-CLOUD (DMZ), une règle de filtrage autorise la communication vers Loki :

Source	Destination	Port	Action	Commentaire
Nextcloud_DMZ	SRV-LOG (10.200.1.80)	3100 TCP	pass	Loki push logs

Structure du projet Ansible

```
ansible-alloy/
├── ansible.cfg                    # Configuration Ansible
├── playbook.yml                   # Playbook principal
├── inventory/
│   └── prod.yml                   # Inventaire des serveurs
└── roles/
    └── alloy/
        ├── tasks/
        │   └── main.yml           # Tâches d'installation
        ├── templates/
        │   └── config.alloy.j2    # Template de config Alloy
        └── handlers/
            └── main.yml           # Handler de redémarrage
```

Configuration complète des fichiers

ansible.cfg

```
[defaults]
inventory = inventory/prod.yml
remote_user = sio
host_key_checking = False

[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
```

inventory/prod.yml

L'inventaire liste tous les serveurs Linux à configurer avec la variable globale loki_url pointant vers le serveur de logs :

```
all:
  vars:
    loki_url: "http://10.200.1.80:3100/loki/api/v1/push"
  children:
    linux:
      hosts:
        SRV-ANS:
          ansible_host: 10.200.1.70
        SRV-CLOUD:
          ansible_host: 192.168.1.1
          collect_docker: true
        SRV-FOG:
          ansible_host: 10.200.1.60
        SRV-GLPI:
          ansible_host: 10.200.1.120
        SRV-XDR:
          ansible_host: 10.200.1.40
        SRV-ZABBIX:
          ansible_host: 10.200.1.45
```

Note : la variable collect_docker: true sur SRV-CLOUD active la collecte des logs Docker en plus des logs système. Les autres serveurs n'ont pas Docker, donc cette variable est absente.

playbook.yml

```
---
- name: Déployer Grafana Alloy sur les serveurs Linux
  hosts: linux
  become: true
  roles:
    - alloy
```

roles/alloy/tasks/main.yml

Ce fichier contient toutes les tâches exécutées sur chaque serveur cible :

```
---
- name: Installer les dépendances
  ansible.builtin.apt:
    name: [curl, gpg]
    state: present
    update_cache: true

- name: Créer le répertoire pour la clé GPG
  ansible.builtin.file:
    path: /etc/apt/keyrings
    state: directory
    mode: "0755"

- name: Ajouter la clé GPG Grafana
  ansible.builtin.shell: |
    curl -fsSL https://apt.grafana.com/gpg.key \
      | gpg --dearmor -o /etc/apt/keyrings/grafana.gpg
    chmod a+r /etc/apt/keyrings/grafana.gpg
  args:
    creates: /etc/apt/keyrings/grafana.gpg

- name: Ajouter le dépôt Grafana
  ansible.builtin.shell: |
    echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] \
      https://apt.grafana.com stable main" \
      > /etc/apt/sources.list.d/grafana.list
  args:
    creates: /etc/apt/sources.list.d/grafana.list

- name: Installer Alloy
  ansible.builtin.apt:
    name: alloy
    state: present
    update_cache: true

- name: Déposer la configuration Alloy
  ansible.builtin.template:
    src: config.alloy.j2
    dest: /etc/alloy/config.alloy
    owner: root
    group: alloy
    mode: "0640"
  notify: Redémarrer Alloy

- name: Ajouter alloy au groupe systemd-journal
  ansible.builtin.user:
    name: alloy
    groups: systemd-journal
    append: true

- name: Ajouter alloy au groupe adm
  ansible.builtin.user:
    name: alloy
    groups: adm
    append: true

- name: Ajouter alloy au groupe docker
  ansible.builtin.user:
    name: alloy
    groups: docker
    append: true
  when: collect_docker | default(false)
```

```
- name: Démarrer et activer Alloy
  ansible.builtin.systemd:
    name: alloy
    state: started
    enabled: true
    daemon_reload: true
```

roles/alloy/handlers/main.yml

```
---
- name: Redémarrer Alloy
  ansible.builtin.systemd:
    name: alloy
    state: restarted
```

roles/alloy/templates/config.alloy.j2

Ce template Jinja2 génère une configuration Alloy adaptée à chaque serveur. Les variables {{ inventory_hostname }}, {{ loki_url }} et {{ collect_docker }} sont remplacées automatiquement par Ansible :

```
// ========================================
// Alloy config — {{ inventory_hostname }}
// ========================================

// ---- Collecte Journal systemd ----
loki.source.journal "systemd" {
  forward_to = [loki.write.default.receiver]
  labels     = {
    host = "{{ inventory_hostname }}",
    job  = "journal",
  }
}

// ---- Collecte fichiers syslog ----
local.file_match "syslog" {
  path_targets = [
    { __path__ = "/var/log/syslog",   job = "syslog" },
    { __path__ = "/var/log/auth.log", job = "auth" },
    { __path__ = "/var/log/kern.log", job = "kernel" },
    { __path__ = "/var/log/dpkg.log", job = "dpkg" },
  ]
}

loki.source.file "syslog" {
  targets    = local.file_match.syslog.targets
  forward_to = [loki.relabel.host.receiver]
}

// ---- Ajout du label host ----
loki.relabel "host" {
  forward_to = [loki.write.default.receiver]
  rule {
    action       = "replace"
    replacement  = "{{ inventory_hostname }}"
    target_label = "host"
  }
}

{% if collect_docker | default(false) %}
// ---- Collecte logs Docker ----
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
}

loki.source.docker "docker" {
  host       = "unix:///var/run/docker.sock"
  targets    = discovery.docker.containers.targets
  forward_to = [loki.relabel.docker.receiver]
}

loki.relabel "docker" {
  forward_to = [loki.write.default.receiver]
  rule {
    action       = "replace"
    replacement  = "{{ inventory_hostname }}"
    target_label = "host"
  }
  rule {
    source_labels = ["__meta_docker_container_name"]
    target_label  = "container"
```

```
  }
}
{% endif %}

// ---- Envoi vers Loki ----
loki.write "default" {
  endpoint {
    url = "{{ loki_url }}"
  }
}
```

Exécution du déploiement

Depuis le contrôleur Ansible (SRV-ANS), lancer :

```
cd ~/ansible-alloy
ansible-playbook playbook.yml
```

Résultat attendu : Ansible se connecte à chaque serveur, installe Alloy depuis le dépôt Grafana, dépose la configuration personnalisée, ajoute l'utilisateur alloy aux groupes nécessaires, et démarre le service.

Vérification post-déploiement

Vérifier que Alloy tourne sur tous les serveurs :

```
ansible all -m shell -a "systemctl status alloy | head -3"
```

Vérifier que les logs arrivent dans Loki :

```
# Lister les labels
curl -s "http://10.200.1.80:3100/loki/api/v1/labels"

# Lister les serveurs qui remontent
curl -s "http://10.200.1.80:3100/loki/api/v1/label/host/values"
```

Résultat attendu :

```
{"status":"success","data":["SRV-ANS","SRV-CLOUD","SRV-FOG","SRV-GLPI","SRV-
XDR","SRV-ZABBIX"]}
```

Configuration manuelle des serveurs Windows

Les serveurs Windows ne sont pas gérés par Ansible (nécessiterait WinRM). L'installation d'Alloy se fait manuellement en PowerShell administrateur sur chaque serveur.

Installation

```
# Télécharger l'installeur (~200 Mo)
Invoke-WebRequest -Uri "https://github.com/grafana/alloy/releases/
  latest/download/alloy-installer-windows-amd64.exe"
  -OutFile "$env:TEMP\alloy-installer.exe"

# Installer en silencieux
Start-Process -FilePath "$env:TEMP\alloy-installer.exe"
  -ArgumentList "/S" -Wait
```

Configuration

Créer le fichier de configuration (adapter le hostname pour chaque serveur) :

```
@"
// Alloy config - DC-IUTO5

loki.source.windowsevent "application" {
  eventlog_name = "Application"
  forward_to    = [loki.relabel.host.receiver]
}

loki.source.windowsevent "system" {
  eventlog_name = "System"
  forward_to    = [loki.relabel.host.receiver]
}

loki.source.windowsevent "security" {
  eventlog_name = "Security"
  forward_to    = [loki.relabel.host.receiver]
}

loki.relabel "host" {
  forward_to = [loki.write.default.receiver]
  rule {
    action       = "replace"
    replacement  = "DC-IUTO5"
    target_label = "host"
  }
}

loki.write "default" {
  endpoint {
    url = "http://10.200.1.80:3100/loki/api/v1/push"
  }
}
"@ | Set-Content -Path "C:\Program Files\GrafanaLabs\
  Alloy\config.alloy" -Encoding UTF8

Restart-Service Alloy
```

Serveurs Windows du projet

Serveur	IP	Logs collectés
DC-IUTO5	10.200.1.1	Application, System, Security

SRV-FIC1	10.200.1.11	Application, System, Security
SRV-FIC5	10.200.1.15	Application, System, Security
SRV-IMP	10.200.1.140	Application, System, Security

Erreurs rencontrées

#	Erreur	Cause	Solution
1	Permission denied (publickey)	Clé SSH non copiée sur les cibles	ssh-copy-id sio@IP sur chaque serveur
2	sudo: not found	sudo non installé sur certains serveurs	apt install -y sudo via su -
3	sudo demande un mot de passe	NOPASSWD non configuré	echo 'sio ALL=(ALL) NOPASSWD: ALL' > /etc/sudoers.d/sio
4	Handler 'Redémarrer Alloy' not found	Fichier handlers/main.yml manquant	Créer le fichier avec le handler de redémarrage
5	relabel_rules = [] : capsule error	Syntaxe River invalide	Supprimer la ligne relabel_rules = []

Commandes utiles

```
# Relancer le déploiement complet
ansible-playbook playbook.yml

# Relancer à partir d'une tâche
ansible-playbook playbook.yml \
  --start-at-task="Déposer la configuration Alloy"

# Vérifier le statut sur tous les serveurs
ansible all -m shell \
  -a "systemctl status alloy | head -3"

# Voir les labels dans Loki
curl -s "http://10.200.1.80:3100/loki/api/v1/labels"

# Voir les hôtes qui remontent
curl -s "http://10.200.1.80:3100/loki/api/v1/label/host/values"

# Dans Grafana (http://10.200.1.80:3000)
# Explore > Loki > {job="journal"}
# Explore > Loki > {host="SRV-CLOUD"}
# Explore > Loki > {container="nextcloud-app"}
```